

MNEMOSYNE

A Graph-Neural-Network Memory System for
Persistent, Cross-Session AI Agents

System Architecture & Technical Documentation

Table of Contents

1. Executive Summary	3
1.1 Who This Document Is For	3
2. What the System Does	4
2.1 The Four Memory Types	4
2.2 Why a Graph Instead of a Flat List	5
2.3 Self-Maintenance: Aging and Forgetting	5
3. System Architecture	6
3.1 Component Responsibilities	6
3.2 Data Storage Responsibilities	7
3.3 Why Qwen Is External, Not Containerized	7
4. The Conversation Lifecycle	8
4.1 Step-by-Step Description	8
5. Memory Write Pipeline	10
5.1 Importance Estimation	10
5.2 Persistence Order and Why It Matters	11
5.3 Embedding Generation	11
5.4 Graph Edge Construction	11
6. Hybrid Retrieval & the Graph Neural Network	12
6.1 Why GraphSAGE	12
6.2 Node Features	13
6.3 Architecture	13
6.4 Dual Supervision and Loss	13
6.5 The Hybrid Scoring Formula	14
6.6 Caching	14
6.7 Cold-Start Behavior	14
7. The Forgetting Service	15
8. Prompt Construction & the Qwen Integration	16
9. API Reference	18
10. Deployment & Operations	20
11. Design Decisions & Lessons Learned	22

1. Executive Summary

Mnemosyne is a persistent memory system for AI agents. It allows a conversational agent to remember facts, preferences, behavioural rules, and past events about a user across sessions, not just within a single chat and to retrieve the right memories at the right time using a combination of vector similarity search and a trained Graph Neural Network (GNN).

Most conversational AI systems either forget everything when a session ends or store the entire conversation history and replay it on every turn. Neither approach scales well: the first produces an agent with no continuity, and the second eventually overflows the language model's context window and buries genuinely important information in noise.

Mnemosyne addresses this with a typed, scored, and graph-connected memory store. Every piece of information the agent learns is classified into one of four types, scored for importance, embedded into vector space, and linked to semantically related memories. At retrieval time, a hybrid ranking algorithm, part classical vector search, part learned graph neural network, selects only the memories that matter for the current query, and a forgetting service prunes memories that have become stale or unused.

The result is an agent that becomes more accurate and more personalized the longer it interacts with a given user, while keeping the amount of information sent to the underlying language model small and relevant.

1.1 Who This Document Is For

- Non-technical readers will find the system's purpose and behaviour described in Sections 1–2 without requiring code-level understanding.
- Developers extending or maintaining the codebase should focus on Sections 3–6, which cover the request lifecycle, data flow, and module responsibilities in implementation detail.
- Readers interested in the machine learning internals, the GraphSAGE architecture, loss functions, and scoring mathematics should read Section 6 and the Technical Appendix in full.

2. What the System Does

At its core, Mnemosyne answers one question every time the agent receives a message: “What do I already know that is relevant to this?” It answers that question by treating memory as a structured, quarriable, and self-maintaining graph rather than an ever-growing transcript.

2.1 The Four Memory Types

Every memory the system stores is classified into exactly one of four types. This typing drives both how a memory is weighted for importance and how it is presented back to the language model.

Type	Definition	Example	Base Weight
Preference	How the user likes things done — tone, format, style.	“User prefers concise answers.”	0.70
Fact	Stable, durable information about the user.	“User works as a software engineer.”	0.60
Episode	A specific event from a past conversation.	“User asked about Python async IO.”	0.40
Rule	A behavioral directive the agent must follow.	“Always give concise answers.”	0.85

These base weights are starting points, not fixed values. The importance score for an individual memory is adjusted upward when its content contains emphatic language (words such as “always,” “never,” “must,” or “important”), and is further refined over time by the GNN's learned relevance score, described in Section 6.

2.2 Why a Graph Instead of a Flat List

A flat list of memories, ranked only by vector similarity to the current query, is the conventional approach and works reasonably well at small scale. Mnemosyne goes further by connecting memories to one another: whenever a new memory is written, it is compared against the user's existing memories, and an edge is drawn to any memory with cosine similarity at or above a fixed threshold of 0.75.

This produces clusters of related memories, for example, several memories about a user's work, several about communication preferences, several about an ongoing project, that the GNN uses as context when scoring relevance. A memory's importance is no longer judged in isolation; it is judged considering what it is connected to.

2.3 Self-Maintenance: Aging and Forgetting

Every conversational turn ages every active memory belonging to that user by incrementing a `turns_since_access` counter. Memories that are retrieved and used have this counter reset to zero. Memories that go a long time without being accessed accumulate age, which lowers their recency boost in retrieval scoring and eventually makes them candidates for archival by the forgetting service.

This mirrors how human memory works in practice: information that is frequently relevant stays sharp and easily recalled, while information that is never needed fades in priority without being abruptly deleted. Archived memories are not destroyed, they are marked inactive and excluded from active retrieval and graph construction, but remain recoverable.

3. System Architecture

Mnemosyne is composed of five layers running as independent Docker containers on a shared local network, plus one external dependency: the Qwen large language model, accessed over HTTPS via Alibaba Cloud's Model Studio. The diagram below shows how a user's message travels from the Streamlit frontend, through the FastAPI orchestration layer and memory engine, out to Qwen, and back.

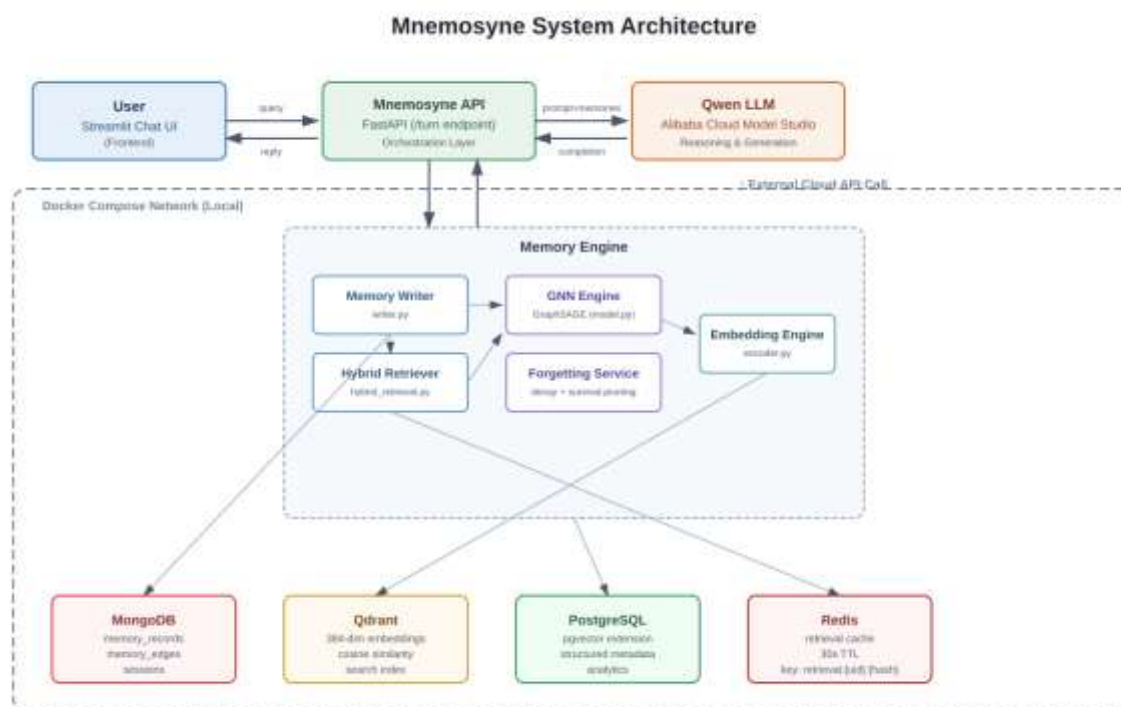


Figure 3.1 — Mnemosyne system architecture: frontend, API layer, memory engine, databases, and the external LLM call.

3.1 Component Responsibilities

Component	Role	Technology
Frontend	Chat interface the user interacts with directly	Streamlit
API Layer	Orchestrates each conversation turn end-to-end	FastAPI
Memory Writer	Classifies, scores, and persists new memories	Python module
Hybrid Retriever	Ranks and selects memories for the current query	Python module + Redis
GNN Engine	Learns relevance and cluster structure over the memory graph	PyTorch Geometric

Forgetting Service	Archives low-value, stale memories	Python module
Embedding Engine	Converts text to vectors and runs similarity search	sentence-transformers + Qdrant
LLM Client	Builds the memory-aware prompt and calls Qwen	OpenAI-compatible SDK

3.2 Data Storage Responsibilities

Four data stores are used, each chosen for a distinct access pattern rather than as redundant general-purpose databases:

- MongoDB holds the canonical memory records, their metadata (importance, access count, age), session state, and the graph edge list connecting related memories.
- Qdrant holds the 384-dimensional embedding vector for every memory and serves fast approximate nearest-neighbor search filtered by user.
- PostgreSQL with the pgvector extension is provisioned for structured analytics and as a secondary vector-capable store; it underpins reporting and longer-term aggregate analysis separate from the hot retrieval path.
- Redis caches the results of recent hybrid retrievals for thirty seconds per user/query pair, avoiding repeated GNN forward passes for repeated or rapid-fire queries.

3.3 Why Qwen Is External, Not Containerized

Every component except the language model itself runs locally in Docker, with no outbound dependency on a cloud database. The single exception is intentional: running a competitive large language model locally is computationally prohibitive for an individual developer, so Mnemosyne calls out to Qwen, hosted on Alibaba Cloud Model Studio, using an OpenAI-compatible REST interface. This is the only network boundary the system crosses, and it occurs once per conversational turn — after memory retrieval has already narrowed the context down to a small, relevant set.

4. The Conversation Lifecycle

This section traces a single conversational turn from the moment the user presses send in the Streamlit interface to the moment a reply appears on screen. Every step below corresponds directly to code executed inside the `/turn` endpoint of the FastAPI application.

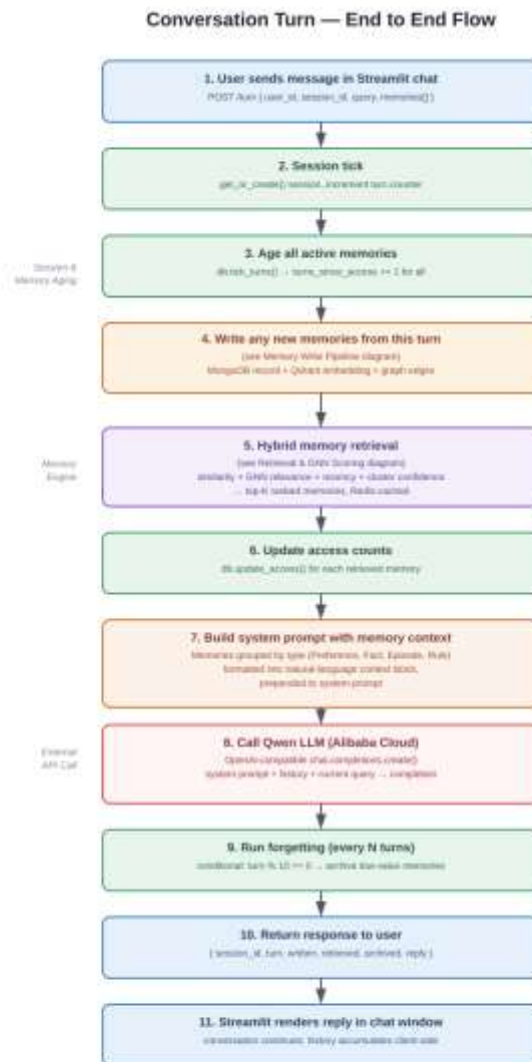


Figure 4.1 — End-to-end flow of a single conversational turn, from user input to rendered reply.

4.1 Step-by-Step Description

1. The user types a message into the Streamlit chat UI, which sends a POST request to /turn containing the user's ID, an optional session ID, the message text, and any explicit new memories to record.
2. The session store retrieves the existing session for this user, or creates a new one, and increments the turn counter for that session. This counter drives the forgetting schedule described in item 9 below.
3. Every active memory belonging to this user has its turns_since_access counter incremented by one. This is the aging step described conceptually in Section 2.3.
4. Any new memories included in the request are written through the full memory write pipeline (Section 5), which classifies, scores, embeds, and graph-connects each one.
5. If the request includes a query, the hybrid retriever (Section 6) is invoked to find and rank the most relevant existing memories for that query.
6. Each retrieved memory has its access count incremented and its age reset to zero — retrieval itself is a signal that a memory is still valuable.
7. The retrieved memories are grouped by type and formatted into a natural-language context block, which is prepended to the system prompt sent to the language model.
8. The LLM client calls Qwen via the OpenAI-compatible chat completions endpoint, passing the memory-aware system prompt, any prior conversation history supplied by the frontend, and the current user message.
9. Every tenth turn, the forgetting service runs automatically, archiving memories whose importance and recency have both decayed below threshold.
10. The API returns a structured response containing the session ID, turn number, IDs of newly written memories, the retrieved memories, any archived memory IDs, and Qwen's reply text.
11. Streamlit renders the reply in the chat window. The frontend is responsible for accumulating conversation history client-side and passing it back on subsequent turns so Qwen retains short-term conversational context within the session, layered on top of Mnemosyne's long-term memory.

5. Memory Write Pipeline

Writing a memory is more than an insert into a database. Each new memory passes through importance estimation, persistence, embedding, and graph linking before it is considered fully indexed and available for retrieval.

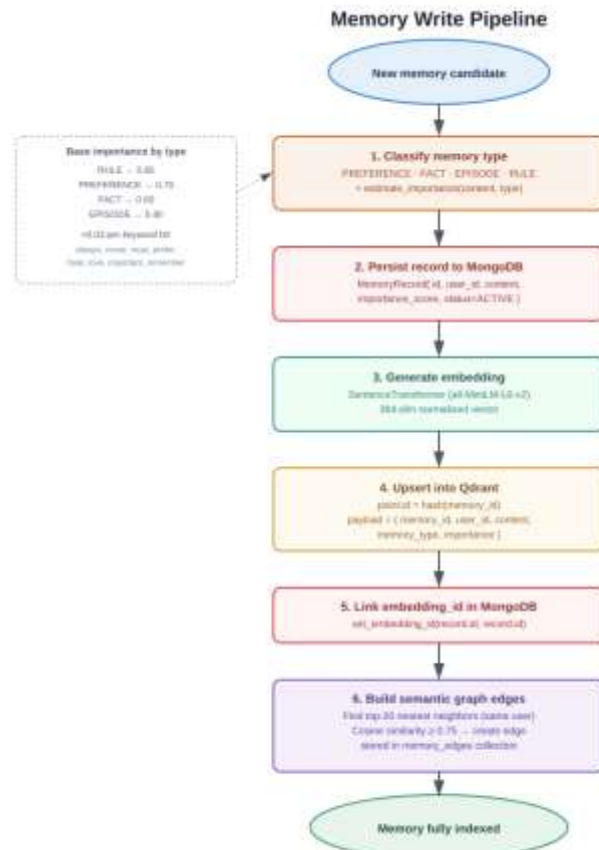


Figure 5.1 — The six-step memory write pipeline, from raw content to a fully graph-connected memory.

5.1 Importance Estimation

Before anything is written to a database, the content and declared memory type are passed through an importance estimator. This function starts from a base score determined purely by memory type (rules score highest by default, since they are explicit behavioral instructions; episodes score lowest, since they describe one-off events that are often safe to forget).

That base score is then increased by a fixed increment for every emphatic keyword found in the memory's content — words such as always, never, must, prefer, hate, love, important, require, every time, and remember each add a small boost, capped so the final importance score never exceeds 1.0. This is intentionally a simple heuristic at write time; the GNN engine described in

Section 6 later refines relevance dynamically based on how the memory actually behaves in the graph, rather than relying on keyword matching alone.

5.2 Persistence Order and Why It Matters

The record is written to MongoDB first, before any embedding work happens, because MongoDB is the source of the memory's canonical ID. That ID is then used as the point identifier in Qdrant and as the join key for the graph edge collection, ensuring all three stores agree on what a given memory's identity is. Only after the embedding is successfully stored in Qdrant does the MongoDB record get updated with a reference back to its own embedding ID, closing the loop.

5.3 Embedding Generation

Text content is converted into a 384-dimensional normalized vector using a sentence-transformers model (all-MiniLM-L6-v2). Normalization means the vectors have unit length, which makes cosine similarity equivalent to a simple dot product, a meaningful performance optimization at scale, since dot products are cheaper to compute than full cosine similarity with separate magnitude normalization on every comparison.

5.4 Graph Edge Construction

After the embedding is stored, the system searches Qdrant for the twenty nearest neighbours to the new memory, scoped to the same user. Any neighbour with cosine similarity at or above 0.75 receives an edge in the `memory_edges` collection, recorded with the similarity score as edge weight. This threshold was chosen empirically: high enough to avoid connecting only superficially related memories, low enough that meaningfully related memories, phrased differently but semantically close, still connect.

These edges are what the GNN consumes as its message-passing structure, described next.

6. Hybrid Retrieval & the Graph Neural Network

Retrieval is the most technically involved part of Mnemosyne, and the part most responsible for the system's claim to “increasingly accurate decisions.” Rather than ranking memories by cosine similarity alone, every retrieval call combines four independent signals into a single hybrid score, one of which, the GNN relevance score, is learned rather than hand-specified.

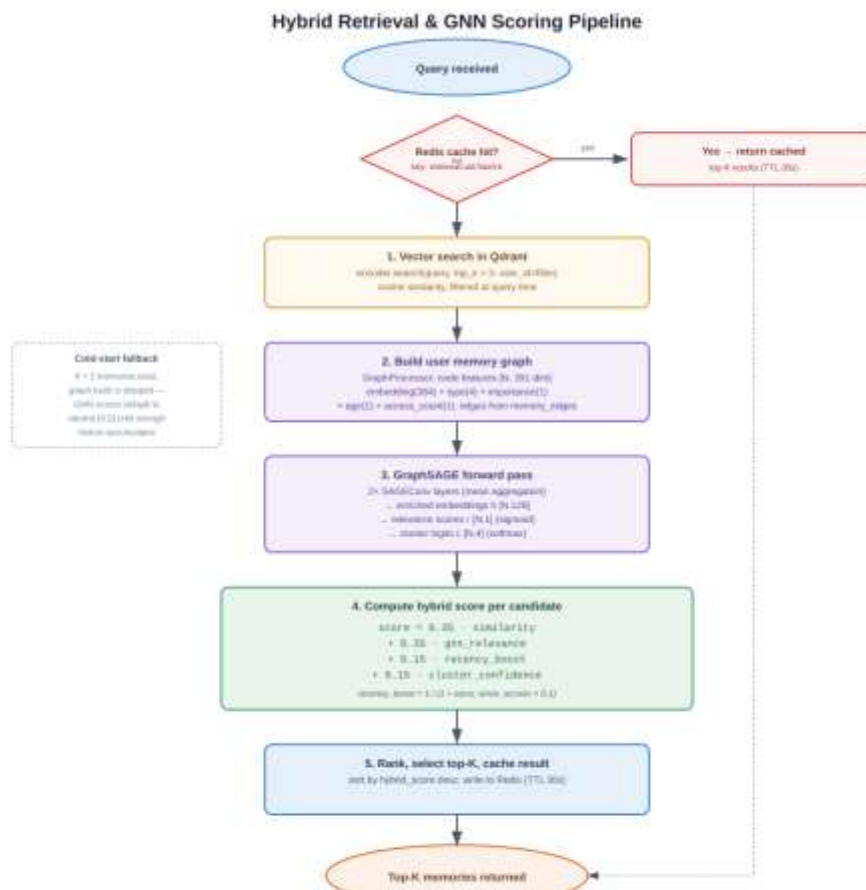


Figure 6.1 — The five-step hybrid retrieval pipeline, including the GraphSAGE forward pass and final scoring formula.

6.1 Why GraphSAGE

A memory's relevance is not just a property of its own content, it is also a property of its neighbourhood. A memory that is densely connected to other frequently accessed, high-importance memories is plausibly more central to the user's actual interests than an isolated memory with a superficially similar embedding. This is precisely the kind of structural signal that a Graph Neural Network is built to learn, and GraphSAGE specifically was chosen because it

generalizes well to graphs that grow over time, new memories (and therefore new nodes) are added continuously, and GraphSAGE's neighborhood-sampling aggregation does not require retraining from scratch when the graph's size changes, unlike some spectral graph methods.

6.2 Node Features

Every memory node entering the GNN is represented as a 391-dimensional feature vector, assembled as follows:

Feature	Dimensions	Description
Embedding	384	The sentence-transformer embedding generated at write time.
Memory type	4	One-hot encoding of preference / fact / episode / rule.
Importance score	1	The write-time importance estimate, clipped to [0, 1].
Normalized age	1	Log-scaled <code>turns_since_access</code> , relative to the user's oldest memory.
Normalized access count	1	Log-scaled <code>access_count</code> , relative to the user's most-accessed memory.

6.3 Architecture

The model consists of two stacked SAGEConv layers using mean aggregation over neighbours, followed by two parallel output heads:

- Layer 1: SAGEConv($391 \rightarrow 256$), followed by ReLU activation and dropout ($p = 0.2$).
- Layer 2: SAGEConv($256 \rightarrow 128$), followed by ReLU activation. The output of this layer is the enriched embedding h , used both for relevance scoring and as a higher-quality representation than the raw input embedding.
- Relevance head: a small feed-forward network ($128 \rightarrow 64 \rightarrow 1$) with a sigmoid output, producing a relevance score in $[0, 1]$ for each memory node.
- Cluster head: a parallel feed-forward network ($128 \rightarrow 64 \rightarrow 4$) producing unnormalized logits over the four memory types, trained as an auxiliary task that encourages the enriched embeddings to remain structurally aware of memory type even after two rounds of neighbourhood aggregation.

6.4 Dual Supervision and Loss

The model is trained with two objectives simultaneously. The relevance head is supervised with binary cross-entropy against a label indicating whether a memory was meaningfully accessed in the relevant time window. The cluster head is supervised with standard categorical cross-entropy against the memory's true type. The two losses are combined as a weighted sum:

```
total_loss = relevance_loss +  $\alpha$  × cluster_loss  
( $\alpha$  = 0.5 by default — balances the two supervision signals)
```

Training the cluster head alongside relevance is a deliberate regularization choice: a model trained on relevance alone can collapse toward shortcuts that ignore memory type structure entirely. Forcing the shared embedding to remain useful for type classification keeps the representation more semantically grounded.

6.5 The Hybrid Scoring Formula

At inference time, four scores are combined for every retrieval candidate:

```
hybrid_score = 0.35 × similarity  
              + 0.35 × gnn_relevance  
              + 0.15 × recency_boost  
              + 0.15 × cluster_confidence  
  
recency_boost = 1 / (1 + turns_since_access × 0.1)
```

Vector similarity and learned relevance are weighted equally and dominate the score, since both are direct, well-calibrated signals of topical fit. Recency and cluster confidence contribute smaller corrective weights — recency rewards memories that have recently proven useful, and cluster confidence rewards memories whose neighbourhood structure is unambiguous, which empirically correlates with cleaner, more reliable memories.

6.6 Caching

Because the GNN forward pass involves rebuilding a user's memory graph and running two rounds of message passing, repeated identical queries within a short window are served from a Redis cache instead, keyed by a hash of the user ID, query text, and requested result count, with a thirty-second time-to-live. Thirty seconds was chosen deliberately short: memory state changes frequently (every write, every access, every forgetting pass), so a long-lived cache risks serving stale rankings.

6.7 Cold-Start Behaviour

A brand-new user, or a user with too few memories to form a meaningful graph, cannot yet benefit from learned relevance scoring. In this case, graph construction is skipped entirely and the GNN relevance and cluster confidence terms default to a neutral 0.5, allowing the hybrid score to degrade gracefully to a vector-similarity-dominated ranking until enough interaction history accumulates for the graph signal to become meaningful.

6.8 Solution to Cold-Start

When a new user arrives, the system doesn't know anything about them, so instead of leaving the GNN with an empty graph, it immediately looks up the closest matching user persona from a pre-built set of archetypes and seeds the graph with 3–5 plausible memories tagged as low-confidence placeholders.

From that point, every time the user sends a message, two scoring systems run in parallel — the GNN scores memories using graph relationships, and flat retrieval scores them using plain similarity. These two scores are blended using a weight called α . Early on α is near zero, so flat retrieval dominates and the unproven seed memories don't mislead the system. As the user generates real memories, α climbs toward 1 and the GNN progressively takes over.

At the same time, the seed memories are quietly decaying — their influence on the GNN score shrinks with every real memory added. By the time the user has had a meaningful number of conversations, the seeds have faded to near nothing and the GNN is running entirely on genuine user data.

The end result is that the user never experiences a hard fallback to dumb retrieval. The system starts cautious, warms up gradually, and hands full control to the GNN only once it has earned the trust.

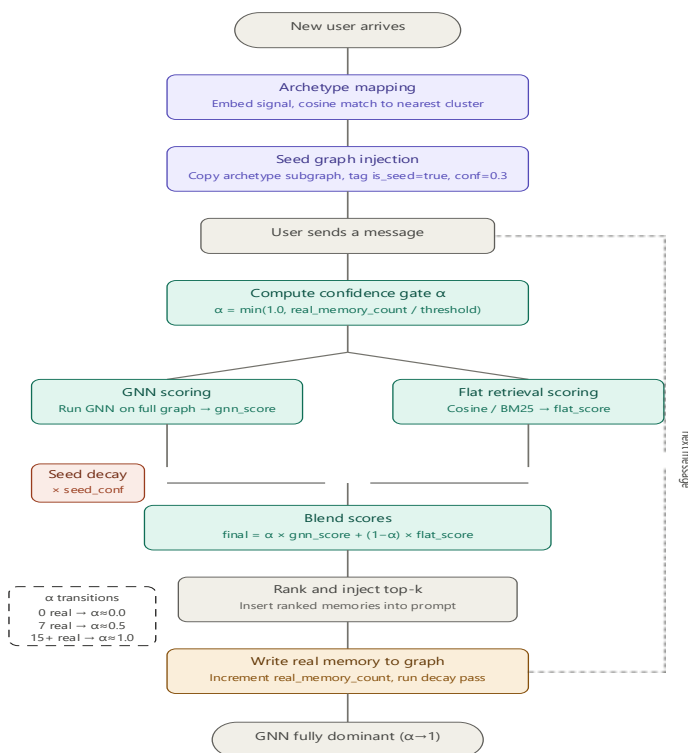


Figure 6.2: Solution to cold start

7. The Forgetting Service

A memory system that never forgets anything eventually suffers the same fate as a system with no memory at all: the signal-to-noise ratio collapses, and retrieval quality degrades as genuinely important memories compete for ranking with thousands of trivial ones. The forgetting service exists to keep the active memory set lean without permanently destroying information.

7.1 When Forgetting Runs

Rather than running on every turn - which would be wasteful, since memory importance does not meaningfully change turn-to-turn, the forgetting pass runs once every ten turns per user, triggered automatically inside the /turn endpoint.

7.2 Archival, Not Deletion

When a memory is selected for forgetting, its status field is changed from active to archived. It is immediately excluded from the active memory list, from future graph construction, and from retrieval candidates, but its underlying record is never deleted from MongoDB. This preserves the option to inspect, restore, or analyse historical memories later without any data loss, and avoids the risk of a forgetting policy being too aggressive and irreversibly discarding something that later turns out to matter.

7.3 Relationship to the GNN Relevance Score

The forgetting service and the GNN's relevance head are conceptually complementary: the GNN scores how relevant a memory is right now, for the purpose of retrieval ranking, while the forgetting service makes a binary decision about whether a memory should remain in the active set at all going forward. A memory can have a moderate GNN relevance score and still be retrieved occasionally, while simultaneously being a long-term forgetting candidate if its importance and access patterns continue to decline.

8. Prompt Construction & the Qwen Integration

The final step before any language model sees a token of text is the construction of a memory-aware system prompt. This is where Mnemosyne's internal memory representation is translated into natural language the LLM can use directly.

8.1 Why Group by Type

Retrieved memories are not simply concatenated into the prompt in ranked order. They are first grouped under clear headings, User Preference, Known Fact, Past Episode, Behavioural Rule, because this gives the language model an explicit signal about how to weight each piece of information. A behavioural rule should be treated as an instruction to follow, not as a passive fact to mention; grouping makes that distinction unambiguous rather than relying on the model to infer it from undifferentiated text.

8.2 System Prompt

Given a set of retrieved memories, the constructed system prompt follows this shape:

```
You are Mnemosyne, an AI agent with persistent memory. Unlike a typical assistant, you do not start fresh each conversation, you carry forward what you have learned about this user across every past session, and that accumulated knowledge is what makes you useful to them.
```

```
# How to use what you remember
```

```
Memories you are given come in four kinds, and they are not all equal:
```

- Rules are binding. They are explicit instructions about how you must behave (e.g. response length, tone, things to avoid). Follow them even if the current request does not mention them, and even if following them makes your answer shorter or plainer than you'd otherwise default to.
- Preferences, Facts, and Episodes are context, not instructions. They tell you who this person is, what they care about, and what has happened before – use them to make your response more relevant and personal, but they do not override what the user is actually asking right now.

```
When memories conflict with each other (e.g. two contradictory preferences stored at different times) or when you only have thin, partial information about something the user references, do not paper over it with false confidence. Acknowledge the gap or the conflict briefly and naturally – the same way a person would say "I might be misremembering, but I thought you mentioned...", then ask if needed rather than guessing silently.
```

If you have no relevant memory at all on something, just say so plainly. Never invent details about the user to sound more familiar than you actually are.

How to sound

Use what you know the way a person with a good memory does – naturally, in the flow of the conversation. Do not narrate that you are "checking memory" or "recalling stored data," and do not announce which memory type something came from. The user should experience continuity, not a database lookup.

8.3 The OpenAI-Compatible Interface

Qwen is accessed through Alibaba Cloud's Model Studio using an endpoint that mirrors the OpenAI chat completions API exactly, which means the integration uses the standard OpenAI Python SDK pointed at a different base URL rather than a bespoke client. This was a deliberate architectural choice: it keeps the LLM client portable, so swapping to a different OpenAI-compatible provider in the future requires changing only configuration, not code.

Setting	Value
Base URL	https://dashscope-intl.aliyuncs.com/compatible-mode/v1
Model (default)	qwen-plus
Interface	OpenAI-compatible <code>chat.completions.create()</code>

9. API Reference

The FastAPI application exposes the following endpoints. All endpoints accept and return JSON.

9.1 POST /turn

The primary endpoint, used for every conversational exchange. Handles session ticking, memory aging, memory writes, retrieval, prompt construction, the Qwen call, and periodic forgetting in a single request.

```
Request:
{
  "user_id": "string",
  "session_id": "string | null",
  "memories": [{ "content": "...", "memory_type": "...", "tags": [] }],
  "query": "string",
  "top_k": 5,
  "history": [{ "role": "user|assistant", "content": "..."}]
}

Response:
{
  "session_id": "...", "turn": 4, "written": [...],
  "retrieved": [...], "archived": [...], "reply": "..."
}
```

9.2 POST /memory/write

Writes a single memory directly, outside the conversational turn flow. Useful for backfilling memories or administrative tooling.

9.3 POST /memory/retrieve

Runs hybrid retrieval directly without going through a full conversational turn, no session ticking, no LLM call. Useful for debugging retrieval quality in isolation.

9.4 GET /memory/list/{user_id}

Returns all active memories for a user with full metadata, including importance score, access count, and age.

9.5 GET /memory/graph/{user_id}

Returns the node list and adjacency structure of a user's memory graph, useful for visualizing or debugging the graph construction layer independently of the GNN.

9.6 GET /sessions/{user_id}

Lists all sessions recorded for a given user.

9.7 GET /health

Simple liveness check used by Docker's health check configuration and by the test suite before running integration tests.

10. Deployment & Operations

10.1 Container Topology

All services are defined in a single `docker-compose.yml` and run on a shared local network: the API container, MongoDB, PostgreSQL with pgvector, Qdrant, and Redis. Only the API container exposes application logic; the four data stores are accessed exclusively through their respective Python clients and are never queried directly by the frontend.

Service	Image	Port
api	custom build (Python 3.11/3.12-slim base)	8000
mongo	mongo:7	27017
postgres	pgvector/pgvector:pg16	5432
qdrant	qdrant/qdrant:v1.12.0	6333
redis	redis:7-alpine	6379

10.2 Configuration

All secrets and environment-specific values — database URLs, the Qwen API key, the selected Qwen model — are supplied via a `.env` file consumed by the API container at startup. No credentials are hardcoded in source.

10.3 Version Pinning

Two version-sensitivity issues surfaced during development and informed a general project policy of explicit version pinning. The Qdrant Python client must stay within one minor version of the Qdrant server itself, or the client raises a compatibility warning and may behave unpredictably; the project pins `qdrant-client==1.12.0` to match the containerized server version exactly. Similarly, PyTorch Geometric's compiled extensions (`torch-scatter`, `torch-sparse`, `torch-cluster`) must be installed from a wheel index matching the exact installed PyTorch version, which the Dockerfile handles explicitly rather than relying on automatic resolution.

10.4 Testing

A full integration test suite exercises the live API — health checks, the write pipeline, direct and conversational retrieval, multi-turn memory persistence across session boundaries, importance scoring, memory aging, graph construction, and session management — by issuing real HTTP requests against the running Docker stack rather than mocking any component. This was a deliberate choice: because the system's value proposition rests on multiple services agreeing with each other (MongoDB's view of a memory matching Qdrant's, the GNN's view of the graph

matching the edge collection), integration-level testing catches a category of bug that unit tests on individual modules would miss entirely.

11. Design Decisions & Lessons Learned

11.1 Why Not LangChain or LangGraph

LangChain and LangGraph are orchestration frameworks: they provide pre-built memory abstractions, chain runners, and agent loops that trade architectural control for speed of prototyping. Mnemosyne deliberately occupies a different layer of the stack, it is memory infrastructure, comparable to the database engine underneath an orchestration framework rather than a replacement for one. LangChain's built-in memory classes (conversation buffers, summary memory, simple vector-store-backed memory) are each a simplified special case of something Mnemosyne implements more completely: typed memory, decay, graph relationships, and learned relevance scoring. The two are not mutually exclusive, a LangGraph agent loop could use Mnemosyne as its memory backend in place of a simpler built-in store.

11.2 Filtering at Query Time, Not After Retrieval

An early version of the retrieval path ran an unfiltered top-K vector search and then filtered results down to the current user afterward. Under multi-user load this silently produced incorrect results: if another user's memories dominated the global top-K for a given query, the current user could receive an empty or incomplete result set even though their own relevant memories existed in the database. The fix was to push the `user_id` filter into the Qdrant query itself, ensuring the similarity search only ever considers vectors belonging to the requesting user in the first place.

11.3 Payload Completeness

A subtle bug arose from the embedding payload stored in Qdrant omitting the memory's raw text content, only metadata was stored alongside the vector. Retrieval worked correctly in the sense that the right memory IDs and similarity scores came back, but any downstream consumer that needed the actual memory text (most notably, the prompt-construction step feeding Qwen) found nothing to display. This highlighted a general principle followed throughout the project: a vector store payload should be self-sufficient for every consumer that will read from it, not just the consumer that was being built at the time the write path was first implemented.

11.4 Graceful Degradation Over Hard Failures

Both the cold-start case (too few memories for a meaningful graph) and a GNN scoring failure are handled by falling back to neutral default scores rather than raising an error that would break retrieval entirely. This reflects a general design philosophy: the learned components of

the system are treated as enhancements layered on top of a reliable vector-similarity baseline, not as load-bearing dependencies that the system cannot function without.

